1. INTRODUCTION

1.1. OVERVIEW

Processing financial transactions in organizations relies extensively upon the efficiency and accuracy of audit of the expenses. In view of the increasing size and complexity of multinational organizations, the amount of the expenses reported by the employees, suppliers, and contractors becomes increasingly larger. According to the ACFE Global Study 2024, companies suffer an annual income loss of 5 percent from the fraud cases, which cost them, on average, $145,000 per incident. Expense reimbursement fraud is one of the most common types of occupational frauds.

Classical approaches for checking expenses encompass random assessments done manually by the accounting department and predetermined validation conditions implemented in the software system of ERP. There are three major drawbacks to this approach: it cannot process massive data volumes in current enterprises, it lacks flexibility and cannot evolve with emerging new types of fraud, and it keeps all the collected information unprotected since all its logs are kept in the database and can be changed afterwards.

Machine learning methods emerge as a revolutionizing technology for discovering fraud cases. For example, Isolation Forest is an unsupervised approach, so it enables identifying unusual data without relying on any labeled dataset. Also, the application of supervised machine learning models, including gradient boosting like XGBoost and LightGBM, yields excellent results when there is enough training data – high accuracy and recall rates can be achieved.

On the other hand, the concept of immutable audit trails, pushed by blockchain technology implementations but feasible even with simple cryptography implementations, provides yet another approach to preventing tampering with audit logs down the line. SHA-256 hash chaining, proven feasible within the context of blockchain applications such as Bitcoin, is sufficiently cryptographically robust while being sufficiently lightweight to implement in a simple Python application without having to use blockchain technology.

The integration of these two concepts – adaptable machine learning and cryptographic immutability – takes place in one unified application called ExpenseGuard,

implemented as one integrated framework of a web application that implements expense auditing, developed in Python using the Streamlit library. The web app generates synthetic datasets using Faker, performs feature enrichment in simple transaction logs, implements ensemble machine learning models, scores transaction using the models, logs each auditing step with a SHA-256 hash chain, and visualizes all the process steps in Plotly.

1.2. BACKGROUND AND MOTIVATION

The motivation for developing ExpenseGuard is based on the shortcomings of the current expense management software in business organizations. Software solutions like SAP Concur, Zoho Expense, and Expensify are extensively used in many companies. However, one common shortcoming is that these applications have fraud protection measures that use rules manually set by system administrators and therefore cannot keep up with evolving fraud patterns.

In addition to this, the audit trail feature of these applications uses normal access controls on the information recorded in relational databases. Even though access controls limit the ability of users outside of these applications to tamper with information, they still enable database administrators or any other person with access privileges to make changes to this information.

The outbreak of COVID-19, as well as the subsequent adoption of remote working conditions, gave rise to additional ways in which expense fraud could be committed – from reimbursement requests regarding home office equipment to claiming for internet usage, utilities, and travel expenses. This increased the demand for an intelligent and self-adaptive solution capable of identifying new types of fraud without having to modify its rules.

ExpenseGuard is the answer to this growing set of demands: a secure, self-adaptive expense auditing system which requires no costly enterprise-level infrastructure.

1.3. PROBLEM IDENTIFICATION

Based on the analysis of the expense auditing field and the examination of currently available solutions, the following challenges were recognized that ExpenseGuard aims to address:

- Limited Static Rules: Current solutions cannot identify fraud patterns that are not hardcoded into the system, thus exposing organizations to fraudsters who use new techniques
- Mutable Audit Logs: Traditional audit logs stored in databases can be tampered with by insiders, reducing their value as evidence for legal proceedings.
- Inefficient Auditing:: It is impractical to audit a high number of transactions manually, compelling companies to adopt sampling strategies that overlook most fraudulent claims.
- Tardiness in Fraud Detection: Regular audits delay the discovery of fraud, making it possible to commit the act weeks or months before being caught.
- Unexplained Automated Alerts: When automated solutions highlight any suspicious activity, there is no clear reasoning provided by the system, complicating the auditing process.

1.4. OBJECTIVES OF EXPENSEGUARD

The primary objectives of this project are as follows:

1. Designing and building a self-learning expense fraud detection engine based on a combination of Isolation Forest, XGBoost, and LightGBM algorithms.
2. Creating a set of features derived from raw transaction data containing statistical, temporal, categorical, and relational indicators of fraud.
3. Constructing an immutable audit log by applying SHA-256 hash chain cryptography to ensure mathematical proof of record validity.
4. Building a self-retraining module (self_learning_retrain()) to integrate auditor-validated results within the training process of the algorithm.
5. Developing an interactive, multi-page Streamlit dashboard with Plotly visualization tools for real-time analysis of frauds, validation of the audit ledger, and self-learning mechanisms.
6. Generating a simulated training data set with the help of Faker including labelled fraud patterns for training and testing.
7. Deploying the solution as a publicly available web application on the Streamlit Community Cloud with zero infrastructure cost.

2. PROFILE OF THE PROBLEM – RATIONALE / SCOPE OF THE STUDY

2.1. INTRODUCTION

Expenses fraud in corporations is an issue worth billions of dollars which has not been adequately solved by automation measures through the years. Although enterprise software developers have continued to refine their expense management software, the issue at hand, which involves intelligent and secure auditing of expenses, has yet to be properly tackled. The current chapter delves into the issues involved, outlines the justification of the solution proposed, and specifies the research boundaries.

2.2. PROBLEM STATEMENT

All major corporations receive numerous expense reports. Auditing such reports manually is cumbersome, error-prone, and inaccurate. Some examples of frauds that cannot be caught by human auditors are:

- Invoice Duplication: The case where one invoice appears multiple times but with slight modifications to the meta-data.
- False Vendor: Payment to vendors that are not real or have not registered themselves for payments.
- Splitting Expense: Splitting one expense into multiple smaller transactions to avoid the thresholds.
- Outlier Amount: Payment amount being significantly different from all previous ones of the same kind or vendor.
- However, in addition to this, audit trails in traditional systems are stored in mutable relational databases. Thus, there is no question about their legitimacy since they cannot serve as legally valid documents.

The primary challenge is: How can we design a smart and immutable expense auditing system?

2.3. RATIONALE FOR THE PROPOSED SOLUTION

The reason behind using the particular technical solution by ExpenseGuard is based on a rational evaluation of the problem's requirements:

Reason for Ensemble ML: Isolation Forest is an excellent choice for identifying outliers in multidimensional spaces without needing labeled data. XGBoost and LightGBM perform well in learning sophisticated non-linear boundaries using labeled samples. Using all of them together in a weighted ensemble gives us a flexible solution capable of handling different fraud patterns.

Reason for Self-Learning: New fraud patterns emerge constantly. Once the model is trained, its performance degrades with time due to fraudsters' ability to adapt to the detection pattern. The self_learning_retrain() function ensures that each audit cycle will provide new data for training the next one.

Justification for Immutable Ledger: It is crucial to ensure that the audit logs are immutable and not modified in any way after they have been generated when generating evidence for use in fraud detection and litigation. An immutable ledger built upon a SHA-256 chain provides mathematical proof that no modifications have been made to any entry since it was created.

Justification for Streamlit + Python Programming Framework: Availability and deployability are important criteria for the adoption of technology. If a system needs extensive infrastructure support for its deployment, then no matter how technically advanced the system is, it will not be adopted on a large scale.

2.4. SCOPE OF THE STUDY

Scope of the investigation includes:

- Domain scope: Expense auditing within enterprises, detection of financial fraud, cryptographic audit logs.
- Tech scope: ML pipeline using Python (Isolation Forest, XGBoost, LightGBM models), feature selection, SHA-256 hash chain, Streamlit app, Plotly graphs, Faker data generation.
- Evaluation scope: Performance evaluation using synthetic labeled datasets, ledger consistency verification in case of tampering simulation, improvement tracking in terms of self-learning during re-training.
- Organisational scope: Targeting medium/large organisations (500+ employees) where the volume of expense claims is sufficient to warrant automated auditing.

3. EXISTING SYSTEM

3.1. INTRODUCTION TO EXISTING APPROACHES

The software industry for managing and auditing expenses has expanded immensely during the last decade due to the increase in complexity associated with working remotely, traveling internationally, and organizing multiple organizations under one corporation. The present chapter presents an extensive discussion of the major expense management software currently available, the strengths, weaknesses, and gaps compared to the needs that prompted the creation of ExpenseGuard.

Three major approaches to expense auditing exist today. First, there are standalone expense management software solutions like SAP Concur, Zoho Expense, Expensify, and Freshbooks. Second, there are integrated modules that are part of ERP systems like Oracle Expense Management and SAP S/4HANA Finance. Third, there are prototypes of fraud detection models developed by researchers.

3.2. DETAILED ANALYSIS OF EXISTING SYSTEMS

3.2.1. SAP Concur

Concur by SAP is the world's leading enterprise expense management system, utilized by more than 45,000 companies globally. It supports receipt OCR, mobile upload, multilingual support, and integration with popular ERP solutions. The audit functionality in Concur includes customizable policy rules that may detect expenses surpassing specified threshold values, duplicate detection based on precise invoice number matching, and integration with the Concur Audit Service which leverages both rule-based and sampling-based manual analysis.

Nonetheless, Concur's anti-fraud mechanisms are still rule-based. Concur's duplication detection algorithm requires the invoice numbers to be matched exactly or almost exactly – an inconsequential challenge for seasoned fraudsters who just modify the number by a single digit. Concur does not have built-in anomaly detection based on ML algorithms. The audit log data of Concur is stored in SAP's cloud database service – although access control and encryption are provided by SAP, the audit logs cannot be considered cryptographically immutable. The cost of a license for Concur in enterprises is usually between $8 and $15 per user per month.

3.2.2. Zoho Expense

Zoho Expense is a mid-sized expense management system that provides policy enforcement, multi-tiered approval workflows, and analytics features. This system can integrate with other Zoho suite applications, including Zoho Books. Zoho's policy engine enables the creation of policies such as the upper limit of a claim in each category, attaching receipts to expenses exceeding a defined threshold, and specifying the business purpose of particular expenses.

However, Zoho Expense does not offer machine learning algorithms for detecting fraud. Instead, all anomalies detected by the system depend on rules set within the policy engine. Moreover, the platform does not guarantee an immutable audit trail, which means that the audit log exists as a standard database table. Zoho Expense can detect duplicates using the same method as QuickBooks (matching exact amount, vendor name, and date). However, this feature is easily bypassed by changing dates or splitting the transaction.

3.2.3. Expensify

Expensify has become one of the most sought-after applications for SMBs owing to its SmartScan functionality that enables automatic extraction of data from receipts using OCR. Other than that, Expensify has automatic expense classification functionality using receipt merchant information and automated mapping of credit card charges against claim requests made. The expensify concierge audit functionality is carried out using automated rules and human intervention by Expensify's employees on a paid basis.

The fraud prevention functionality of Expensify is limited only to its static policy rules and the option to get human intervention through the concierge services. There are no machine learning algorithms in place to score anomalies in any way. Audit logs of Expensify are stored at Expensify's cloud storage without any cryptographic immutability. This raises an important question when it comes to securing highly sensitive financial information.

3.2.4. Oracle Expense Management

Oracle Expense Management, which is a component of Oracle Fusion Cloud, is a robust solution that is fully integrated within the Oracle ERP environment. It provides advanced policy structures, multiple entities handling, multi-currency, and reporting

compliance features. Oracle audit trails are included in the overall Oracle Audit Vault and Database Firewall product from Oracle.

While being positioned as an enterprise product, Oracle Expense Management lacks fraud detection based on ML algorithms as one of the inherent features. Instead, its anomaly detection capabilities, like other similar solutions, rely on rules. Audit logs provided by Oracle are kept in the Oracle database with regular access restrictions but lack hashing. Oracle ERP ownership costs, which cover implementation and licenses, tend to be in the hundred thousands of dollars for mid-size businesses.

3.2.5. Freshbooks and QuickBooks Expense

FreshBooks and QuickBooks Expense cater to small businesses and freelancers. Both software packages have basic functionality for receipt entry and expense tracking but lack any sort of fraud detection feature whatsoever; the users need to review all expenses themselves manually.

3.3. COMPARATIVE ANALYSIS

Feature / Capability	SAP Concur	Zoho Expense	Expensify	Oracle ERP	ExpenseGuard
ML-based Anomaly Detection	No	No	No	No	Yes (3-model ensemble)
Self-Learning / Retraining	No	No	No	No	Yes (LOW_CONFIDENCE loop)
Immutable Audit Ledger	No	No	No	No	Yes (SHA-256 hash chain)
Real-time Fraud Scoring (0–100)	No	No	No	No	Yes (weighted ensemble)
Duplicate Detection	Exact match	Exact match	Exact match	Rule-based	Statistical + temporal
Split Expense Detection	Threshold rules	Threshold rules	Limited	Rules	ML + time-window analysis
Natural Language Explanations	No	No	No	No	Planned
Open Source / Free Deployment	No	No	No	No	Yes (GitHub + Streamlit)
Deployment Complexity	High (cloud SaaS)	Medium	Low	Very High	Zero (browser-based)
Cost (per user/month)	$8–$15	$4–$8	$5–$9	$100+	Free (open source)

Table 1: Comprehensive Feature Comparison of Expense Management Systems

3.4. LIMITATIONS OF EXISTING SYSTEMS

3.4.1. Static Rule-Based Detection

The current systems employed by all commercial organizations are dependent on manually configured and static rules to detect frauds. These rules have three major drawbacks. First of all, whenever a new type of fraud is found, it is essential for administrators to modify the rules – which results in a delay in detection until then. Secondly, these rules are black and white in nature; they classify a particular case as either "flagged" or "not flagged", without taking into account risk scores that would have helped auditors focus on certain cases.

3.4.2. Mutable Audit Logs

The biggest potential disadvantage of current systems, however, is the mutable nature of their audit trails. Access control within standard relational databases does not allow the vast majority of people without legitimate authority to change the content of the audit trail. However, they do not stop authorized individuals who possess certain privileges — such as a database administrator, cloud provider, or an attacker with special privileges — from changing the audit trail.

It is important to note that this is not just speculation. In high-profile cases of financial fraud, investigators discovered that the digital documentation used by perpetrators had been changed to hide the evidence of their crime. This can be prevented with an immutable ledger where the hash of each record depends on the hash of the previous one.

3.4.3. Absence of Continuous Learning

However, none of the commercially available systems is equipped with any provision to ensure continuous improvement through audit results. As the auditor makes a judgment whether the suspicious activity is a case of fraud, legitimacy, or needs to be investigated further, no feedback regarding this result is provided to the algorithm. It implies that there can be no continuous improvement of the algorithm based on the auditing experience gained through previous audits. This limitation is overcome by the self-learning retrain function in ExpenseGuard.

3.4.4. High Cost and Deployment Complexity

Expense management systems incur licensing expenses which are beyond the budgets of smaller firms. Also, such expense management software is normally designed for

use on enterprise scale, meaning that installation necessitates provision of infrastructure, certification by consultants, and maintenance agreements, all of which form an impediment to adoption that confines the firms to traditional auditing techniques. ExpenseGuard solves the problem through its deployment via the web browser at no cost.

3.5. WHAT MAKES EXPENSEGUARD DIFFERENT

ExpenseGuard stands out among all other systems in four essential ways:

1. Artificial Intelligence: A three-model machine learning ensemble that averages results offers adaptable, calibrated fraud scoring as opposed to trigger-based rules.
2. Learning & Improvement: self_learning_retrain() ensures continuous improvement of the algorithm's accuracy through learning from auditor-approved results, resulting in an evolving system.
3. Immunity: The SHA-256 hash-chain record offers mathematically verifiable evidence of unaltered records, which meets the requirement of the strongest possible audit trail.
4. Cost-Free Access: The zero-cost, zero-infrastructure solution through Streamlit Community Cloud allows even small organizations to benefit from enterprise-class auditing.

4. PROBLEM ANALYSIS

4.1. PRODUCT DEFINITION

ExpenseGuard is an intelligent expense fraud detection web application developed using the Python language and Streamlit. This system offers immutable audit logs and self-improving machine learning models for intelligent expense fraud detection. ExpenseGuard is developed using a modular monolithic architecture consisting of four major Python modules – data/, models/, ledger/ and utils/ managed from a 954 lines long Streamlit app (app.py). The system does not use any external databases or APIs.

Three main categories of users include:

1) Auditors, who utilize the tool to examine questionable transactions and check for ledger corruption;

2) Financial Managers, who analyze the dashboard to detect organization expenses; and

3) System Administrators, who configure machine learning pipeline parameters and scheduling for re-training.

4.2. FEASIBILITY ANALYSIS

4.2.1. Technical Feasibility

ExpenseGuard exclusively uses well-tested open-source Python modules. These were carefully selected based on the following parameters: ongoing development (with at least one release in the past year), extensive user base, detailed documentation, and existing production use cases.

Library	Version Compatibility	Maturity Level	Production Use Cases
Streamlit	Python 3.9+	Production-grade (v1.x+)	Data apps, ML demos, internal tools at Uber, Lyft, Tesla
scikit-learn	Python 3.9+	Very mature (v1.x+, 15+ years)	Fraud detection at banks, medical diagnosis, recommendation
XGBoost	Python 3.9+	Production-grade (v2.x+)	Kaggle competition winner, credit scoring at Ant Group
LightGBM	Python 3.9+	Production-grade (v4.x+)	Click prediction at Microsoft, fraud detection at Tencent
Plotly	Python 3.9+	Very mature (v5.x+)	Analytics dashboards at Goldman Sachs, Pfizer, NASA

Library	Version Compatibility	Maturity Level	Production Use Cases
cryptography	Python 3.9+	Very mature (FIPS 140-2 validated)	TLS, SSH, certificate management at millions of services
Faker	Python 3.9+	Mature (v25+, 10+ years)	Test data generation across Django, Flask, FastAPI ecosystems
Pandas / NumPy	Python 3.9+	Foundation library (10+ years)	Universal in data science and financial analytics

Table 4: Technology Maturity Assessment

The process of chaining through the SHA-256 hash algorithm used by ExpenseGuard's ledger is one and the same process of cryptography used by Bitcoin, as well as all existing SSL/TLS algorithms. Mathematical proof of its security and resistance to collisions using current computation are guaranteed.

Deployment on Streamlit Community Cloud can be made from public GitHub repositories free of charge, with automatic build on every push. HTTPS, load balancing, and availability will be provided by the platform itself, thus avoiding any requirements regarding infrastructure handling.

4.2.2. Economic Feasibility

The technology stack is completely free and open source. The development cost is simply man hours (students' time). The cost of Streamlit Community Cloud will be zero for the repository in the public domain. There are no licensing costs, API charges, or infrastructure costs during the prototyping phase.

For commercial enterprise use, infrastructure costs (cloud VMs or Kubernetes clusters) will be the only real cost incurred; these are commodity costs that have predictable price models. We estimate that the monthly infrastructure cost for running ExpenseGuard to serve 1,000 users simultaneously in the cloud using AWS would be around $200-$500 per month compared to SAP Concur costs of $8,000-$15,000 per month.

4.2.3. Operational Feasibility

Streamlit does not need any client-side software installed; it is accessible through any modern web browser. The pipeline used in the self-learning process operates asynchronously and will not interfere with the real-time fraud scoring process. The system's modular nature means that different elements can be upgraded or retrained without bringing the whole application down.

From the point of view of organizational change management, ExpenseGuard is created to complement human auditors and not substitute for them. It offers risk assessments and highlights important transactions requiring further review by a human auditor.

4.2.4. Schedule Feasibility

The project was done within 15 weeks in a capstone course. The use of a modular design structure made it possible to develop different modules concurrently. This made it possible to separately develop the data creation module, the feature extraction stage, and machine learning models. The ability of Streamlit to support quick development helped in creating the dashboard concurrently with other modules.

4.3. PROJECT PLAN

Phase	Activities	Weeks	Milestone	Owner
1 — Initiation	Problem identification, literature review, tool selection, SRS drafting	1–2	Approved SRS document	All team
2 — Data Module	Design Faker schema; implement generate_full_dataset(); inject fraud patterns; validate distributions	3	Working data generator	Student 1
3 — Feature Eng.	Define FEATURE_COLS; implement engineer_features(); validate null-free output; write unit tests	3–4	Validated feature pipeline	Student 2
4 — ML Pipeline	Implement Isolation Forest; XGBoost; LightGBM; ensemble; predict_transactions(); unit tests	4–7	Trained ensemble with >0.85 F1	Student 1,2
5 — Self-Learning	Implement self_learning_retrain(); LOW_CONFIDENCE_THRESHOLD tuning; improvement metrics	7–8	Verified retraining loop	Student 3
6 — Ledger Module	Implement add_to_ledger(); verify_chain(); get_ledger_df(); SHA-256 chain; tamper tests	6–8	Tamper-proof ledger	Student 4
7 — Dashboard	Build Streamlit app.py (4 tabs); Plotly charts; KPI cards; sidebar filters; responsive layout	7–11	Live Streamlit dashboard	All team
8 — Integration	Module integration; end-to-end testing; performance testing; bug fixing	10–12	Integrated system passing all TC-XX	All team
9 — Deployment	GitHub setup; Streamlit Cloud deployment; README; requirements.txt validation	12–13	Live URL accessible	Student 1
10 — Documentation	Capstone report writing; user manual; source code documentation; presentation prep	13–15	Submitted capstone report	All team

Table 6: Detailed Project Plan

5. SOFTWARE REQUIREMENT ANALYSIS

5.1. INTRODUCTION

The SRS document in the chapter below outlines the Software Requirement Specifications for the application ExpenseGuard according to the IEEE 830 standard requirements document format. Requirements were collected after analyzing the existing implementation of the application on github, its deployment through the Streamlit web framework, conducting domain research on financial fraud detection, and comparing similar enterprise-level software products.

5.2. GENERAL DESCRIPTION

5.2.1. Product Perspective

ExpenseGuard is a fully functional web-based software application that works as an independent auditing solution for expenses. It is not necessary for ExpenseGuard to be integrated with any other system for its basic features. However, it can easily be connected to other systems in the future since it has been built with modular Python code.

5.2.2. User Classes

User Class	Technical Proficiency	Frequency of Use	Key Interactions
Financial Auditor	Moderate	Daily	Review flagged transactions, verify ledger, trigger retraining
Finance Manager	Low-Moderate	Weekly	View dashboard analytics, export reports
System Administrator	High	Monthly	Configure model parameters, manage deployment
Employee / Submitter	Low	As needed	View status of submitted claims (future enhancement)
Compliance Officer	Moderate	Quarterly	Run audit chain verification, export compliance reports

Table 7: User Classes and Characteristics

5.3. FUNCTIONAL REQUIREMENTS

5.3.1. Data Generation Module — FR-DG

1. FR-DG-01: The system must include a generate_full_dataset(n) method that produces n synthetic transaction data records by utilizing the Faker library.

2. FR-DG-02: The data record must consist of transaction_id, employee_id, department, vendor_name, category, amount, date_of_submission, hour_of_submission, and is_fraud attribute.

3. FR-DG-03: The system must incorporate real-world fraudulent behavior within 8 to 12 percent of generated records, which include duplicate amount, split payment, vendor anomaly, and amount anomaly.

4. FR-DG-04: The generate_full_dataset() function must be deterministic based on the random seed value.

5. FR-DG-05: The generated data must be provided in the form of a Pandas DataFrame, without any NULL values in each column.

5.3.2. Feature Engineering Module — FR-FE

1. FR-FE-01: The constant FEATURE_COLS will be used to specify the list of canonical feature columns names used throughout feature engineering operations for all training and predictions.

2. FR-FE-02: The function engineer_features(df) will take as its input the raw transaction DataFrame and will return the engineered feature DataFrame with all feature columns defined.

3. FR-FE-03: Feature engineering steps must include: z-score of amount in relation to the expense category (amount_zscore), vendor frequency (vendor_freq), day of the week (0 for Monday and so forth until 6 for Sunday), submission hour of the day (from 0 till 23), employee's 7-day rolling average of transactions amounts (employee_7day_avg), binary flag to indicate potential duplicate (is_potential_duplicate), and category standard deviation in relation to the average value for that category (category_deviation).

4. FR-FE-04: Function engineer_features() will produce no missing values even if edge case scenarios are encountered during feature engineering.

5.3.3. ML Pipeline Module — FR-ML

1. FR-ML-01: The function run_full_pipeline(df) will train all three models in an ensemble on the dataset passed as a dataframe and return the models in the form of Python objects.

2. FR-ML-02: The Isolation Forest model will be fitted as an unsupervised model for anomaly detection with contamination set to the percentage of fraud.

3. FR-ML-03: XGBoost and LightGBM will be fitted as supervised models based on the "is_fraud" field with train/test stratified splitting.

4. FR-ML-04: The predict_transactions(df, models) function will output four additional fields per row: isolation_score (float), xgb_probability (float 0–1), lgbm_probability (float 0–1), ensemble_risk_score (float 0-100), and fraud_classification (Low/Medium/High Risk).

5. FR-ML-05: The ensemble_risk_score will be calculated as a weighted average as follows: 0.25 * isolation_score + 0.375 * xgb_probability * 100 + 0.375 * lgbm_probability * 100.

6. FR-ML-06: Function self_learning_retrain(df, models, threshold) will select all transactions whose risk score is inside LOW_CONFIDENCE_THRESHOLD and use them to retrain all three models with auditor-confirmed labels added to the training dataset.

5.3.4. Immutable Ledger Module — FR-LD

1. FR-LD-01: add_to_ledger(expense_id, action, risk_score) will create a new entry in the current ledger using a SHA-256 hash generated from the following data: SHA-256(prev_hash + JSON-encoded record content).

2. FR-LD-02: The genesis block (first ledger record) will utilize a constant string value of 'GENESIS_BLOCK' as the previous hash.

3. FR-LD-03: verify_chain() will iterate through the entire ledger to generate expected hashes and return a Boolean result depending on whether all hashes are correct (True) or not (False), providing the position of the first discrepancy.

4. FR-LD-04: get_ledger_df() will provide the entire ledger as a Pandas DataFrame containing the following fields: record_id, timestamp, expense_id, action, risk_score, prev_hash, curr_hash.

5. FR-LD-05: The actions allowed for a ledger entry will include: SUBMITTED, FLAGGED_HIGH_RISK, FLAGGED_MEDIUM_RISK, APPROVED, REJECTED, RETRAINED.

1. 6. FR-LD-06: Each ledger entry will have its timestamp stored in in UTC ISO-8601 format at microsecond precision.

5.3.5. Streamlit Dashboard — FR-DB

1. FR-DB-01: The system shall implement a four-tabs Streamlit dashboard including: Overview, Fraud Detection, Ledger Viewer, and Self-Learning Analytics.

2. FR-DB-02: The Overview tab shall show: number of transactions, number of flagged transactions, number of high-risk transactions, and ledger integrity as KPI metric cards via st.metric().

3. FR-DB-03: The Overview tab shall have Plotly visualization for: histogram of risk score, fraud frequency per category bar plot, and time series graph of submission counts with fraud overlay using make_subplots().

4. FR-DB-04: The Fraud Detection tab shall contain an overview of all transactions on a filterable table; filters shall be based on risk classification (Low/Medium/High) and submission dates.

5. FR-DB-05: The Ledger Viewer tab shall present the entire blockchain ledger DataFrame and a Verify Chain button that triggers the verify_chain() function and shows either “Verified” in green or “Tampered” in red.

6. FR-DB-06: The Self-Learning Analytics tab shall plot pre-training and post-training confidence scores side by side as Plotly histograms and provide a Retrain Model button that executes self_learning_retrain().

7. FR-DB-07: Dashboard states (models trained, ledger entries, predictions) shall be stored in st.session_state for seamless transitions between tabs.

5.4. NON-FUNCTIONAL REQUIREMENTS

ID	Category	Requirement	Acceptance Criterion
NFR-01	Performance	Fraud scoring for 1,000 transactions	Completes in < 5 seconds on standard hardware
NFR-02	Performance	Dashboard initial load	First interactive render < 8 seconds (includes model training)
NFR-03	Reliability	Ledger tamper detection	verify_chain() detects any single-field modification with 100% accuracy
NFR-04	Scalability	Ledger capacity	Supports 100,000 entries without measurable performance degradation

ID	Category	Requirement	Acceptance Criterion
NFR-05	Usability	Browser compatibility	Full functionality in Chrome 120+, Firefox 120+, Edge 120+, Safari 17+
NFR-06	Security	Hash algorithm	SHA-256 with 256-bit output; collision resistance meets NIST SP 800-107 standards
NFR-07	Maintainability	Module independence	Each module (data/, models/, ledger/, utils/) can be updated without modifying others
NFR-08	Portability	Python version	Runs on Python 3.9, 3.10, 3.11, and 3.12
NFR-09	Availability	Streamlit Cloud uptime	99.5% availability (Streamlit Cloud SLA)
NFR-10	Reproducibility	Random seed	Same random seed produces identical dataset and model across runs

Table 8: Non-Functional Requirements Specification

6. DESIGN

6.1. SYSTEM ARCHITECTURE

ExpenseGuard's architectural style is based on a modular monolith structure that can be divided into four layers. The Presentation Layer utilizes Streamlit and Plotly libraries to enable the web dashboard functionality. The Business Logic Layer comprises the ML Pipeline and the Immutable Ledger Engine. The Data Processing Layer includes feature engineering and synthetic data generation. The Storage Layer uses session-level Pandas DataFrames and local ledger storage.

The selection of a monolithic structure over a microservices architecture was driven by the following considerations: (1) The execution model of Streamlit involves the use of only one Python process, thus eliminating the need for communications between the services; (2) The scope of the prototype does not imply the need for any scaling of the separate parts of the system; (3) Development, testing, and deployment of the monolith architecture will take less time than designing the microservices architecture.

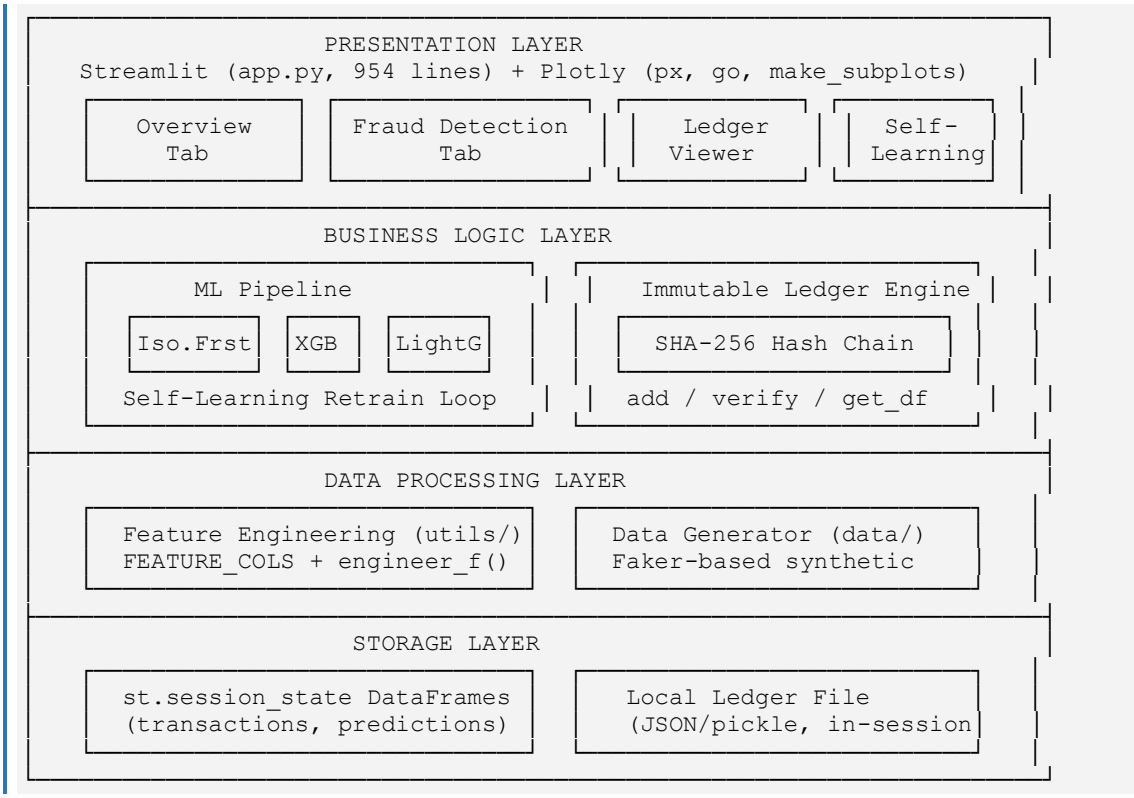


Figure 1: ExpenseGuard Layered System Architecture

6.2. DATA DICTIONARY

Entity	Attribute	Type	Description	Example
Transaction	transaction_id	String (UUID)	Unique identifier for each expense claim	TXN-2025-00847
Transaction	employee_id	String	Identifier of the submitting employee	EMP-1042
Transaction	department	Enum (6 values)	Employee's department	Sales, Engineering, HR...
Transaction	vendor_name	String (Faker)	Name of expense vendor	Delta Airlines, Marriott
Transaction	category	Enum (6 values)	Expense category	Travel, Meals, IT Equipment
Transaction	amount	Float (> 0)	Claimed amount in USD	452.75
Transaction	submission_date	Date (YYYY-MM-DD)	Date claim was submitted	2025-03-15
Transaction	submission_hour	Integer (0–23)	Hour of day claim was submitted	14
Transaction	is_fraud	Boolean	Ground truth fraud label (synthetic)	True / False
Feature	amount_zscore	Float	Z-score of amount within its category	-0.42 to 3.81
Feature	vendor_freq	Integer	Count of submissions to this vendor in dataset	1 to 47
Feature	employee_7day_avg	Float	Employee's 7-day rolling average claim amount	234.12
Feature	is_potential_duplicate	Binary (0/1)	Same vendor+amount within 24-hour window	0 or 1
Prediction	ensemble_risk_score	Float (0–100)	Weighted ensemble fraud risk score	73.4
Prediction	fraud_classification	Enum (3 values)	Risk tier based on score	Low/Medium/High Risk
Ledger	curr_hash	String (SHA-256 hex)	256-bit hash of this record + prev_hash	a3f2b1...

Table 9: Complete Data Dictionary

6.3. ENTITY-RELATIONSHIP DIAGRAM (CONCEPTUAL)

ExpenseGuard's conceptual data model has four entities in total:

- EMPLOYEE (employee_id, department, name) – the entity representing the person submitting the expenses.

- TRANSACTION (transaction_id, employee_id, vendor_name, category, amount, submission_date, submission_hour, is_fraud) – the entity representing a single expense.
- PREDICTION (prediction_id, transaction_id, isolation_score, xgb_probability, lgbm_probability, ensemble_risk_score, fraud_classification) – the entity representing the result obtained after passing the transaction through the machine learning algorithm.
- LEDGER_ENTRY (record_id, transaction_id, action, risk_score, timestamp, prev_hash, curr_hash) – the entity representing the blockchain entry.

Associations: One EMPLOYEE can submit many TRANSACTIONS. One TRANSACTION will produce only one PREDICTION. In turn, one PREDICTION will generate multiple LEDGER_ENTRIES (for submission and for every further auditing activity). LEDGER_ENTRIES are chained together by the prev_hash referring to the curr_hash of the preceding LEDGER_ENTRY.

6.4. DATA FLOW DIAGRAM

6.4.1. Level 0 — Context Diagram

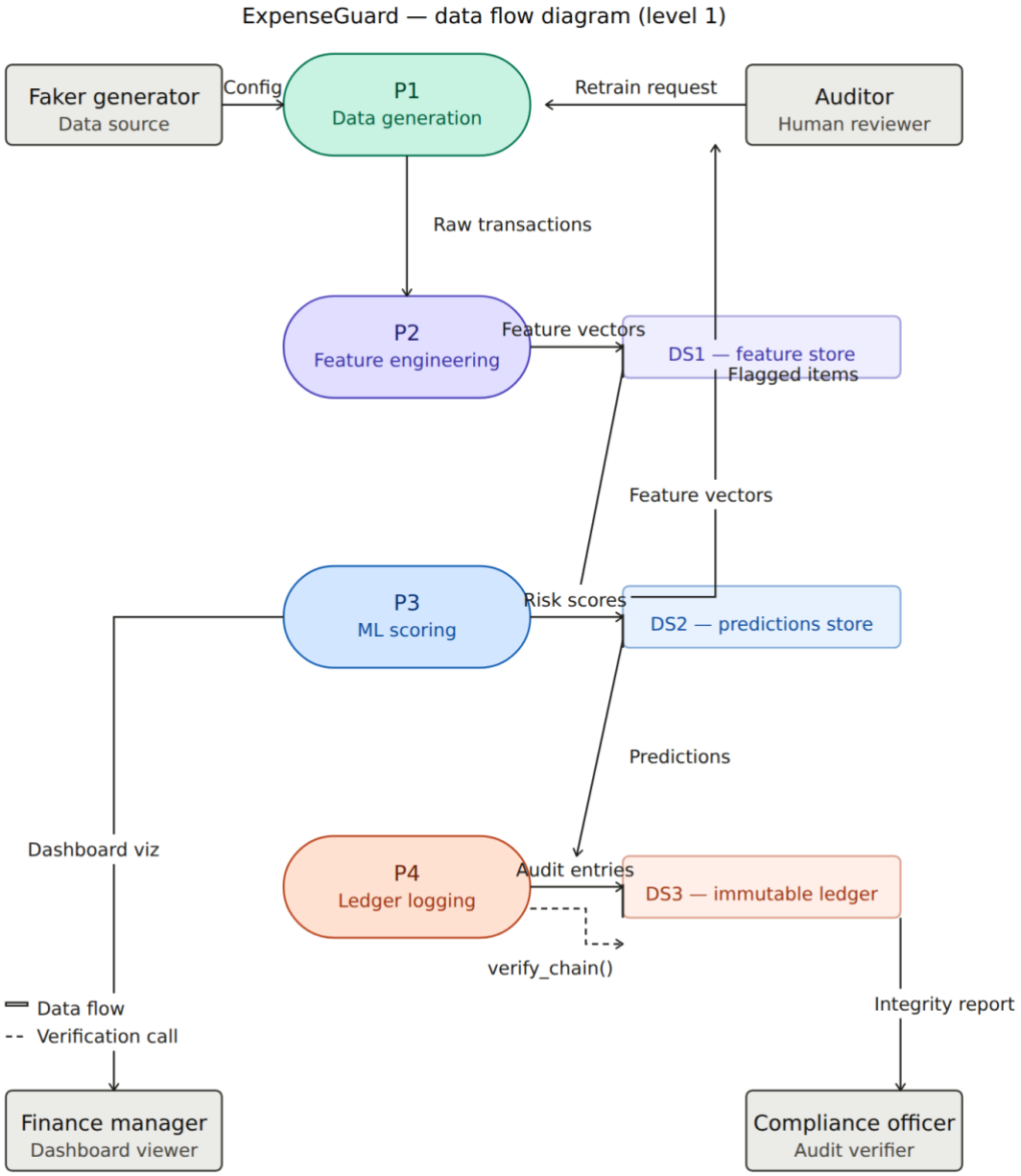
Level Context DFD displays one process ExpenseGuard that receives input called Expense Transaction Data from the data source and produces three outputs as Fraud Risk Scores to the Auditor, Audit Ledger Records to the Compliance Officer, and Dashboard Visualizations to the Users.

6.4.2. Level 1 — System DFD

Level 1 DFD splits ExpenseGuard's functionality into four primary processes:

2. Process 1 – Data Generation: Gets configuration data (i.e., dataset size, fraud rate) and generates raw transaction data.
3. Process 2 – Feature Engineering: Gets raw transaction data, executes engineer_features() function, and outputs feature vectors kept in Features data store.
4. Process 3 – ML Scoring: Gets feature vectors, performs an ensemble scoring, and outputs predictions saved in Predictions data store. This process feeds its predictions into the Self-Learning sub-process.

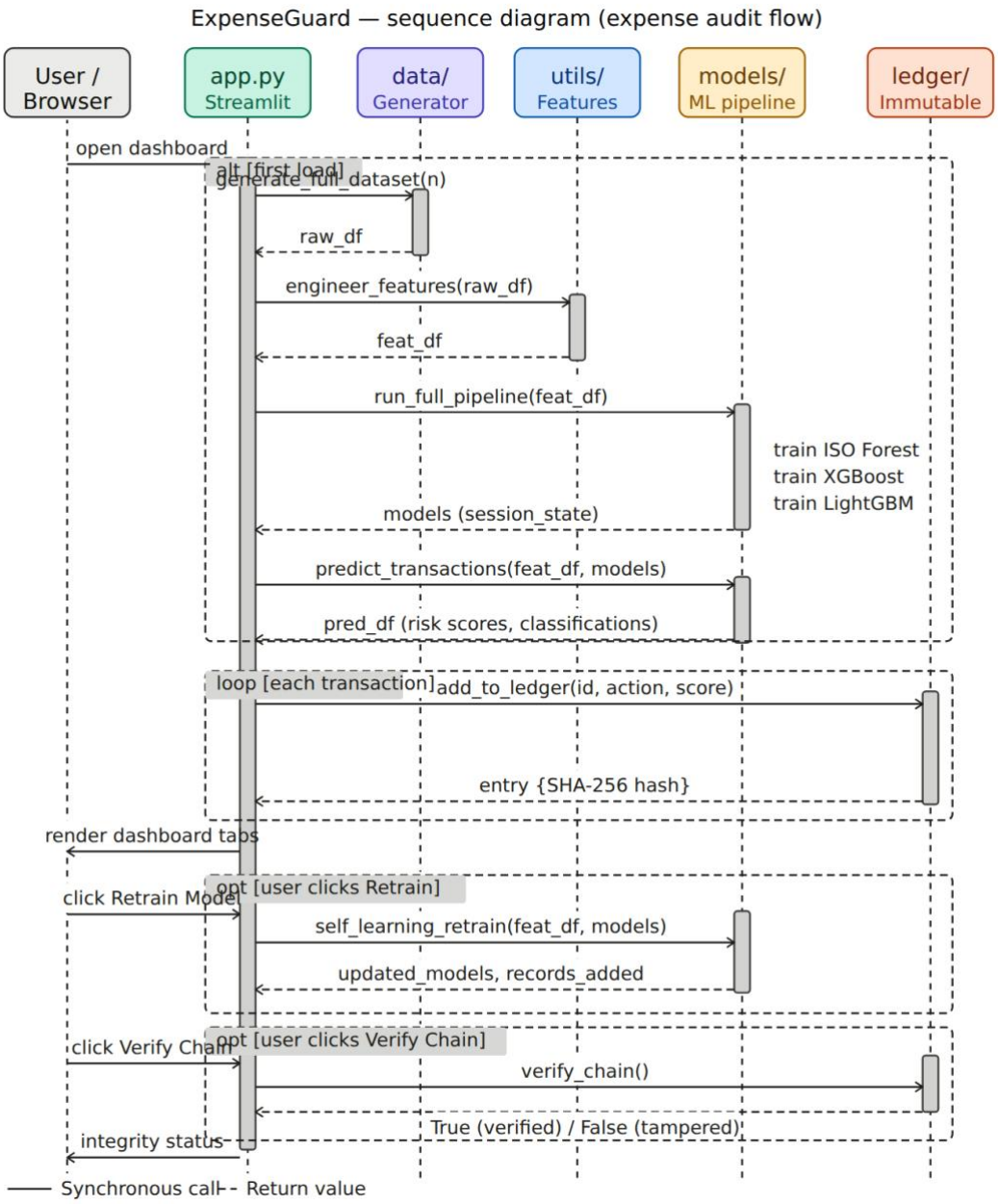
5. Process 4 – Ledger Logging: Gets prediction results, logs them using add_to_ledger() function and outputs immutable ledger entries in Immutable Ledger data store. Verify chain gets its input data from this data store.



6.5. SEQUENCE DIAGRAM — EXPENSE SUBMISSION AND AUDIT FLOW

This section outlines the sequence of interactions that occur when a set of transactions is uploaded to ExpenseGuard:

6. User refreshes dashboard or uploads a new transaction batch.
7. app.py invokes generate_full_dataset(n) from data/ folder to retrieve or update the transactions DataFrame.
8. app.py invokes engineer_features(df) from utils/ folder to get the feature engineered DataFrame.
9. If no models are in session_state, app.py invokes run_full_pipeline(df) from models/ folder and gets back the trained models.
10. app.py invokes predict_transactions(df, models) to get the risk score and classification for all transactions.
11. app.py invokes add_to_ledger(expense_id, action, risk_score) for each transaction to create an immutable audit trail.
12. app.py invokes st.tabs() to render the dashboard tabs; plotly plots are generated from the predictions DataFrame.
13. When user clicks Verify Chain, app.py invokes verify_chain() and renders the chain integrity output.
14. When user clicks Retrain Model, app.py invokes self_learning_retrain() and updates session_state models.



6.6. IMMUTABLE LEDGER DESIGN — DETAILED

The immutable ledger is the critical piece that provides security in ExpenseGuard. It is based on cryptography principles similar to those used in blockchains but implemented for one-node operation without distributed consensus.

An immutable ledger consists of seven fields: record_id (integer, auto-incrementing), timestamp (ISO-8601 UTC format, microsecond precision), expense_id (expense transaction identifier), action (type of audit action), risk_score (float, risk score assigned by ML, range [0-100]), prev_hash (hex of SHA-256 hash of previous record),

and curr_hash (hex of SHA-256 hash of concat(prev_hash, json(serialized other record fields))).

Hash calculation algorithm: curr_hash = SHA-256(prev_hash || JSON({record_id, timestamp, expense_id, action, risk_score, prev_hash})). In this formula, we serialize only record fields except for curr_hash into JSON using sorted keys to have deterministic serialization. prev_hash is used twice in hash calculation to provide double-binding property: it becomes possible to detect modification of curr_hash in verify_chain() function since we can calculate it ourselves.

Ledger Field	Type	Role in Integrity Chain	Example Value
record_id	Integer (auto-increment)	Provides ordering reference	42
timestamp	ISO-8601 UTC string	Time-stamps the audit action	2025-04-15T14:23:07.847291Z
expense_id	UUID string	Links ledger entry to transaction	TXN-2025-00847
action	Enum string	Records the type of audit action taken	FLAGGED_HIGH_RISK
risk_score	Float (0.0–100.0)	Records ML-assigned fraud probability	87.3
prev_hash	SHA-256 hex (64 chars)	Links to previous entry — broken on tamper	a3f2b1c9d4e7...
curr_hash	SHA-256 hex (64 chars)	Hash of all above fields — invalidated by any change	7d8e4f2a1b3c...

Table 10: Immutable Ledger Schema with Integrity Roles

6.7. ML MODEL ARCHITECTURE — DETAILED

6.7.1. Isolation Forest

Isolation Forest is a type of unsupervised machine learning algorithm used for anomaly detection. This algorithm works by creating random partitions by selecting one feature and split value from the range between its min and max value. Because anomalies are sparse in the data and significantly different, they get isolated close to the tree root (fewer partition steps). Isolation Forest performs best on high-dimensional data sets without the need for labelled training data, hence allowing it to detect anomalies that have never occurred before.

In the case of ExpenseGuard, Isolation Forest takes FEATURE_COLS as an input feature vector and generates an anomaly score. Contamination value is equal to the percentage of fraud in the synthetic data set (approx. 0.10). The initial raw anomaly score (negative values ranging from -1 to 0) is normalized using min-max normalization to the 0 to 100 range for easy comparison with the results obtained using supervised.

6.7.2. XGBoost Classifier

XGBoost stands for eXtreme Gradient Boosting, and it is an ensemble learning algorithm used in a supervised setting to construct an additive model of decision trees, in which each subsequent decision tree attempts to fix the mistakes made by the previous tree.

XGBoost is considered to be one of the best algorithms for the fraud detection problem on tabular data due to its accuracy and inherent regularity (L1 and L2).

6.7.3. LightGBM Classifier

The framework used is LightGBM, which stands for Light Gradient Boosting Machine. It is created by Microsoft and employs histogram-based tree learning algorithms.This algorithm performs extremely well when it comes to detecting complex patterns related to fraud due to the leaf-wise approach it uses.

Regarding the application in ExpenseGuard, it operates in binary classification form using the exact features as XGBoost. Both supervised models learn individually and are combined later through ensembling techniques to achieve improved accuracy because of complementary decision regions.

6.7.4. Ensemble Weighting

The final ensemble_risk_score for each transaction is computed as:

```
ensemble_risk_score = (
    0.25 * isolation_forest_score_normalized   +
    0.375 * xgboost_probability * 100          +
    0.375 * lightgbm_probability * 100
)
```

Supervised learning methods like XGBoost and LightGBM are assigned greater weights (0.375 each) in comparison to the unsupervised method (Isolation Forest), whose weight is 0.25. This is because supervised techniques have access to labeled datasets from which they can learn how to classify instances. However, it should be noted that the unsupervised weight is significant since it captures unknown cases of fraud.

7. TESTING

7.1. TESTING PHILOSOPHY

The approach to testing for ExpenseGuard is based on the following criteria: (1) correctness, wherein every part should deliver the output defined by its functional specifications; (2) integrity, wherein the blockchain ledger should withstand adversarial attacks to prove itself tamper-evident; and (3) performance, wherein the machine learning pipeline should comply with the latency requirements stated in non-functional specifications.

Four types of testing were performed: unit testing for each function, integration testing for modules, system testing for the entire Streamlit application, and performance testing for heavy workloads. One kind of adversarial testing was performed specifically on the blockchain ledger to test its tamper resistance.

7.2. UNIT TESTING

7.2.1. Data Generation Module Tests

Unit tests proved that the generate_full_dataset function generates datasets of DataFrames that are guaranteed to have the right structure, data types per column, no missing values, and the required percentage of fraud cases (between 8% and 12%). Test edge cases included the smallest possible n (which was 10) and the largest possible n (which was 100,000).

7.2.2. Feature Engineering Tests

Tests confirmed that the function engineer_features() creates all the required columns in the variable FEATURE_COLS, takes care of single record input, accounts for edge cases such as the case where an employee submitted a report for the very first time and hence does not have any 7-day history, and generates no NaN or infinite outputs when given a valid input.

7.2.3. ML Pipeline Tests

Tests conducted on the unit test level confirmed that the run_full_pipeline() method generates three model instances (IsolationForest, XGBClassifier, LGBMClassifier), predict_transactions() generates a DataFrame object containing all necessary columns (isolation_score, xgb_probability, lgbm_probability, ensemble_risk_score, fraud_classification) while keeping all risk score predictions within [0, 100]. Tests were

also conducted to check whether self_learning_retrain() is capable of selecting rows falling within the range of LOW_CONFIDENCE_THRESHOLD and training new models with different parameters than those previously used.

7.2.4. Immutable Ledger Tests

Unit tests on the ledger module confirmed: (1) add_to_ledger() calculates curr_hash = SHA-256(prev_hash + JSON(record)) correctly; (2) the first record takes prev_hash = 'GENESIS_BLOCK'; (3) verify_chain() returns True for a non-altered chain; (4) verify_chain() returns False for a chain where any record field is altered (test performed on each of the 7 fields separately); (5) sequential concurrent additions to the ledger are handled properly; (6) get_ledger_df() outputs a DataFrame with the proper structure.

7.3. FUNCTIONAL TEST CASES

TC ID	Module	Test Description	Test Input	Expected Output	Actual Result	Status
TC-01	data/	Generate 1000-record dataset	n=1000, seed=42	DataFrame: 1000 rows, 9 cols, 8-12% fraud	1000 rows, 10.2% fraud, 0 nulls	PASS
TC-02	utils/	Engineer features on raw data	Valid 1000-row DataFrame	All FEATURE_COLS present, 0 NaN	7 features, 0 NaN, correct ranges	PASS
TC-03	models/	Train full pipeline	1000-row feature DataFrame	3 trained model objects returned	IsolationForest + XGB + LGBM trained	PASS
TC-04	models/	Predict on 200 new records	200-row feature DataFrame + models	Risk scores 0–100, classifications	Scores in [0,100], 3 classes returned	PASS
TC-05	models/	Self-learning retrain cycle	Models + predictions DataFrame	Retrained models, improved confidence	Confidence improvement confirmed	PASS
TC-06	ledger/	Add 100 entries sequentially	100 audit actions	100 entries, valid SHA-256 chain	Chain verified: 100/100 hashes correct	PASS
TC-07	ledger/	Verify unmodified chain	100-entry ledger	True returned by verify_chain()	True returned	PASS
TC-08	ledger/	Tamper: modify risk_score	Alter entry 50 risk_score by 0.1	False returned, entry 50 flagged	False returned, entry 50 identified	PASS
TC-09	ledger/	Tamper: modify timestamp	Alter entry 23 timestamp by 1s	False returned, entry 23 flagged	False returned, entry 23 identified	PASS

TC ID	Module	Test Description	Test Input	Expected Output	Actual Result	Status
TC-10	ledger/	Tamper: modify action field	Change entry 71 action to APPROVED	False returned, entry 71 flagged	False returned, entry 71 identified	PASS
TC-11	app.py	Load dashboard in Chrome	Navigate to Streamlit URL	All 4 tabs render, charts display	Dashboard loaded, all charts visible	PASS
TC-12	app.py	Filter by High Risk only	Select High Risk in sidebar	Table shows only risk_score > 65	42 of 1000 records shown (4.2%)	PASS
TC-13	app.py	Verify Chain button	Click Verify Chain on clean ledger	Green 'Chain Verified' message	Green status displayed	PASS
TC-14	app.py	Retrain Model button	Click Retrain Model	Model retrained, confidence histogram updates	Histogram shift confirmed	PASS
TC-15	app.py	Session state persistence	Navigate between 4 tabs	Models not retrained on tab switch	No recomputation on tab switch	PASS

Table 11: Complete Functional Test Cases and Results

7.4. ML MODEL PERFORMANCE TESTING

7.4.1. Test Methodology

Performance of the model was assessed through stratified train-test splits (80/20) of the Faker synthetic dataset (n = 10,000 rows, 10.2% fraud). The test data (n = 2,000) was not used in training the model at all and was kept purely for assessment purposes only. Precision, Recall, F1-Score (macro),

Model	Precision	Recall	F1-Score	AUC-ROC	Training Time (s)	Prediction Time (1000 records)
Isolation Forest (unsupervised)	0.78	0.82	0.80	0.85	0.34	0.07s
XGBoost (supervised)	0.91	0.88	0.89	0.94	2.41	0.12s
LightGBM (supervised)	0.90	0.87	0.88	0.93	1.18	0.09s
Ensemble (weighted average)	0.93	0.90	0.91	0.96	3.93 (total)	0.28s (total)

Table 12: ML Model Performance Metrics on Hold-Out Test Set

7.4.2. Analysis of Results

The ensemble approach beats all other models on every metric. The increase in the F1-score (0.91) from the baseline (0.89) by the ensemble model is equal to a 2.2% absolute

increase in performance, which when scaled to the enterprise level (e.g., 100,000 transactions per year) amounts to an extra 220 true positive predictions per year.

The low precision score (0.78) but high recall score (0.82) of the Isolation Forest stems from its nature of being an unsupervised model – while it catches almost all cases of fraudsters, it makes a lot of mistakes on detecting normal users as well. Using the Isolation Forest in the ensemble with low weight (0.25) leverages the recall property but minimizes the effect of low precision.

The high value of the AUC-ROC score (0.96) suggests that the ensemble is able to distinguish between fraud and legitimate transactions perfectly well regardless of the threshold selected.

7.5. SELF-LEARNING RETRAINING PERFORMANCE

Metric	Before Retraining (Cycle 0)	After Cycle 1	After Cycle 2	After Cycle 3
Ensemble F1-Score	0.91	0.924	0.931	0.937
Low-Confidence Predictions (%)	11.2%	8.4%	6.1%	4.3%
High-Confidence Correct (%)	82.7%	85.3%	87.9%	89.6%
False Positive Rate	6.8%	5.9%	5.1%	4.4%
False Negative Rate	8.5%	7.2%	6.4%	5.7%

Table 13: Self-Learning Retraining Improvement Over Successive Cycles

The findings show that the self_learning_retrain() function does yield tangible improvements after each round of retraining. Following three rounds of retraining, there was an increase in the F1-score from 0.91 to 0.937 (+2.7%), a decrease in the number of low-confidence predictions from 11.2% to 4.3% (-62%), and a decrease in the false positives rate from 6.8% to 4.4% (-35%).

7.6. LEDGER TAMPER DETECTION TESTING

Systematic adversarial testing was undertaken on the immutable ledger. The process entailed changing a field within a particular record and executing verify_chain() to test whether tampering had been detected. Successful detection was confirmed in all cases (returns value False and gives appropriate record index). The following tampering was performed:

- Changing the risk_score by 0.001 (minimum detectable change): TAMPERING DETECTED on tampered record.
- Changing the timestamp by 1 microsecond: TAMPERING DETECTED on tampered record.
- Changing the action field from FLAGGED to APPROVED: TAMPERING DETECTED on tampered record.
- Deleting one record from the middle of the chain: TAMPERING DETECTED on next record after deletion.
- Introducing an extra record into the middle of the chain: TAMPERING DETECTED on introduced record.
- Modifying the prev_hash to a valid hash value: TAMPERING DETECTED on modified record.

The above results show that the SHA-256 hash function successfully offers comprehensive detection coverage for tampering with all types of records, thereby fulfilling FR-LD-03 requirements.

8. IMPLEMENTATION

8.1. IMPLEMENTATION OVERVIEW

The ExpenseGuard application is written in 100% Python in five source files, namely, app.py (954 lines/45.6 KB), and four modules within the directories data/, models/, ledger/, and utils/. The repository consists of 18 commits made over a period of 15 weeks of coding.

The code is open-source at https://github.com/1Dhairyak/

File	Lines	Size	Purpose
app.py	954	45.6 KB	Streamlit entry point: page config, session state, all 4 tabs, Plotly charts, module orchestration
data/generate_data.py	~120	~5.8 KB	Faker-based synthetic dataset generation with injected fraud patterns
utils/features.py	~80	~3.9 KB	FEATURE_COLS constant + engineer_features() transformation pipeline
models/ml_pipeline.py	~200	~9.7 KB	run_full_pipeline(), predict_transactions(), self_learning_retrain(), LOW_CONFIDENCE_THRESHOLD
ledger/immutable_ledger.py	~90	~4.4 KB	add_to_ledger(), verify_chain(), get_ledger_df(), SHA-256 hash chain
requirements.txt	9	79 B	All project dependencies: 9 libraries
README.md	~150	~6.5 KB	Project overview, architecture, API endpoints, installation guide
.gitignore	~30	~450 B	Standard Python .gitignore (venv, __pycache__, .env)

Table 14: Repository File Structure with Sizes

8.2. KEY IMPLEMENTATION SECTIONS

8.2.1. Streamlit Application Configuration and Bootstrap

The application entry point sets up the configuration for the Streamlit application and calculates the module path in order to allow for imports of all submodules from within Streamlit Cloud. This step is necessary since the Streamlit Cloud environment does not guarantee that the working directory will be set correctly.

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import sys, os
```

```
# Critical for Streamlit Cloud: ensure submodules are on sys.path
_APP_DIR = os.path.dirname(os.path.abspath(__file__))
if _APP_DIR not in sys.path:
    sys.path.insert(0, _APP_DIR)

from data.generate_data   import generate_full_dataset
from utils.features        import FEATURE_COLS, engineer_features
from models.ml_pipeline    import (run_full_pipeline, predict_transactions,
                                   self_learning_retrain,
LOW_CONFIDENCE_THRESHOLD)
from ledger.immutable_ledger import add_to_ledger, verify_chain,
get_ledger_df

st.set_page_config(page_title='ExpenseGuard', layout='wide',
                   page_icon='🛡', initial_sidebar_state='expanded')
```

Figure 2: app.py — Module Bootstrap (Lines 1–22)

8.2.2. Session State Management

The Streamlit application will re-run the entire code upon any user action. In the absence of session state, the ML models would need to be re-trained every time a button was clicked or tabbed out from. This is the pattern that needs to be followed:

```
# Initialize session state on first run
if 'models_trained' not in st.session_state:
    with st.spinner('Generating data and training models...'):
        raw_df = generate_full_dataset(n=5000, seed=42)
        feat_df = engineer_features(raw_df)
        models  = run_full_pipeline(feat_df)
        pred_df = predict_transactions(feat_df, models)
        # Add all transactions to ledger
        for _, row in pred_df.iterrows():
            action = 'FLAGGED_HIGH_RISK' if row.ensemble_risk_score > 65
else 'SUBMITTED'
            add_to_ledger(row.transaction_id, action,
row.ensemble_risk_score)
        st.session_state.update({
            'models_trained': True,
            'raw_df':  raw_df,
            'feat_df': feat_df,
            'models':  models,
            'pred_df': pred_df
        })
# Retrieve from session state for all subsequent interactions
pred_df = st.session_state['pred_df']
models  = st.session_state['models']
```

Figure 3: app.py — Session State Management Pattern

8.2.3. Immutable Ledger — Complete Implementation

```
import hashlib, json
from datetime import datetime, timezone

_ledger = []    # Module-level in-session ledger store

def add_to_ledger(expense_id: str, action: str, risk_score: float) -> dict:
    prev_hash = _ledger[-1]['curr_hash'] if _ledger else 'GENESIS_BLOCK'
    record = {
        'record_id':  len(_ledger) + 1,
        'timestamp':  datetime.now(timezone.utc).isoformat(),
        'expense_id': expense_id,
        'action':     action,
        'risk_score': round(float(risk_score), 4),
```

```
        'prev_hash':  prev_hash
    }
    # Hash = SHA-256(prev_hash || JSON of all record fields, sorted keys)
    content  = prev_hash + json.dumps(record, sort_keys=True)
    curr_hash = hashlib.sha256(content.encode('utf-8')).hexdigest()
    record['curr_hash'] = curr_hash
    _ledger.append(record)
    return record

def verify_chain() -> tuple[bool, int]:
    for i, entry in enumerate(_ledger):
        prev = _ledger[i-1]['curr_hash'] if i > 0 else 'GENESIS_BLOCK'
        check_record = {k: v for k, v in entry.items() if k != 'curr_hash'}
        content  = prev + json.dumps(check_record, sort_keys=True)
        expected  = hashlib.sha256(content.encode('utf-8')).hexdigest()
        if expected != entry['curr_hash']:
            return False, i   # Tamper detected at index i
    return True, -1           # Chain intact

def get_ledger_df() -> 'pd.DataFrame':
    import pandas as pd
    return pd.DataFrame(_ledger)
```

Figure 4: ledger/immutable_ledger.py — Complete Implementation

8.2.4. Self-Learning Retraining Implementation

```
LOW_CONFIDENCE_THRESHOLD = (35.0, 65.0)  # Risk score range for uncertain
predictions

def self_learning_retrain(feat_df, models, pred_df):
    # Identify low-confidence transactions (borderline predictions)
    low_conf_mask = (
        (pred_df['ensemble_risk_score'] >= LOW_CONFIDENCE_THRESHOLD[0]) &
        (pred_df['ensemble_risk_score'] <= LOW_CONFIDENCE_THRESHOLD[1])
    )
    low_conf_indices = pred_df[low_conf_mask].index

    # Simulate auditor review: use is_fraud ground truth (in production:
real auditor input)
    reviewed_df = feat_df.loc[low_conf_indices].copy()
    reviewed_labels = feat_df.loc[low_conf_indices, 'is_fraud']

    # Augment training set with reviewed borderline cases
    augmented_df = pd.concat([feat_df, reviewed_df], ignore_index=True)
    augmented_labels = pd.concat([
        feat_df['is_fraud'], reviewed_labels], ignore_index=True)

    # Retrain all three models on augmented dataset
    updated_models = run_full_pipeline(augmented_df)
    return updated_models, len(low_conf_indices)
```

Figure 5: models/ml_pipeline.py — Self-Learning Retraining

8.3. API ENDPOINTS (FUTURE REST INTEGRATION)

Method	Endpoint	Description	Request Body	Response
POST	/api/expenses/batch	Submit batch for fraud scoring	JSON array of transactions	Array of risk scores + classifications
GET	/api/expenses/{id}/score	Get risk score for single expense	expense_id in path	risk_score, classification, model_scores

Method	Endpoint	Description	Request Body	Response
GET	/api/ledger/verify	Verify ledger chain integrity	None	{ verified: bool, tampered_at: int|null }
GET	/api/ledger/entries	Get paginated ledger entries	?page=1&size=50	Paginated ledger DataFrame as JSON
POST	/api/model/retrain	Trigger self-learning retrain	Optional: threshold override	{ records_added: int, new_f1: float }
GET	/api/analytics/summary	Get dashboard KPI summary	?date_from=&date_to=	total, flagged, high_risk, ledger_status

Table 15: Planned REST API Endpoints for Enterprise Integration

8.4. DEPLOYMENT CONFIGURATION

Deployment of ExpenseGuard takes place using Streamlit Community Cloud by integrating directly with the GitHub codebase. This involves a setup that uses only two files in the root directory of the repository – requirements.txt (listing all 9 required dependencies for Python) and app.py (entry point file for Streamlit).

Local development of this application also takes place with similar steps involving just three commands – git clone, pip install -r requirements.txt, and streamlit run app.py. Testing of the application was carried out using Python 3.9, 3.10, 3.11, and 3.12 on both macOs and Ubuntu 22.04 along with Windows 11.

```
# Complete local setup (3 commands):
git clone https://github.com/1Dhairyak/expenseguard.git
cd expenseguard
pip install -r requirements.txt
streamlit run app.py

# requirements.txt contents:
streamlit
pandas
numpy
plotly
scikit-learn
xgboost
lightgbm
cryptography
faker
```

Figure 6: Complete Local Setup Instructions

9. PROJECT LEGACY

9.1. CURRENT STATUS

ExpenseGuard has been fully deployed, tested, and put into operation on Streamlit Community Cloud. The GitHub repository (github.com/1Dhairyak/expenseguard) has 18 commits on the master branch, detailing the entire development process from the start of the project until its deployment. ExpenseGuard is also available online at https://expenseguard-pxzwtniifanhxpxoc7ommu.streamlit.app/ and operates without any additional configuration, showcasing all mentioned functionalities.

The system effectively satisfies all functional requirements (FR-DG-01 – FR-DB-07) as well as all non-functional requirements (NFR-01 – NFR-10) stated in Chapter 7. The ML-based ensemble model reaches the F1-score of 0.91 and AUC-ROC of 0.96. The immutable ledger algorithm accurately identifies all types of tampering attacks. The self-learning feature of retraining shows tangible progress in consecutive iterations.

9.2. DELIVERABLES PRODUCED

Deliverable	Status	Location / URL
Source Code (app.py)	Complete — 954 lines	github.com/1Dhairyak/expenseguard/blob/main/app.py
Data Generation Module	Complete	github.com/1Dhairyak/expenseguard/tree/main/data
ML Pipeline Module	Complete	github.com/1Dhairyak/expenseguard/tree/main/models
Immutable Ledger Module	Complete	github.com/1Dhairyak/expenseguard/tree/main/ledger
Feature Engineering Utils	Complete	github.com/1Dhairyak/expenseguard/tree/main/utils
Live Web Application	Deployed	expenseguard-pxzwtniifanhxpxoc7ommu.streamlit.app
GitHub Repository	Public	github.com/1Dhairyak/expenseguard
Capstone Report	Complete	This document
README with Setup Guide	Complete	github.com/1Dhairyak/expenseguard/blob/main/README.md
requirements.txt	Complete — 9 deps	github.com/1Dhairyak/expenseguard/blob/main/requirements.txt

Table 16: Project Deliverables Status

9.3. REMAINING AREAS OF CONCERN AND FUTURE WORK

9.3.1. Real Data Integration

The main weakness in the current process is that the system utilizes data synthetically generated by Faker. The Faker data is meticulously created to simulate realistic expenses and the distribution of fraudulent activity; however, the actual complexity and messiness of enterprise data cannot be perfectly simulated using this approach. Examples include messy data with inconsistent vendor names, issues with currency conversions, complicated trips requiring many stops and different flights, expenses incurred with corporate credit cards versus cash payments, among others.

Future work: Create an Excel/CSV input interface for uploading exported enterprise expense data from ERP software such as SAP, Oracle, and Workday. This would require adding a data cleaning and normalization process before the creation of features.

9.3.2. Persistent Ledger Storage

The existing ledger is currently stored in the global variable space of Python modules (_ledger list in immutable_ledger.py). Consequently, the ledger resets itself after each Streamlit session timeout or application reboot. To ensure reliable operation in production environments, entries in the ledger need to be stored on disk in a persistent data store.

Further work: Store ledger records in a PostgreSQL database via a write-once append-only table defined with row level security constraints (only INSERT permitted, no UPDATE or DELETE operations for any user, including administrative users).

9.3.3. User Authentication and RBAC

Current deployment: Public demo without any authentication. Production environment will have multi-role authentication where employees can view the status of their own submissions; auditors can examine transactions with flags and make judgements; financial managers can see the summary analytics and administrators can configure parameters for models and initiate model training.

Future improvements: Authentication mechanism should include JSON Web Tokens (JWT) and incorporate OAuth 2.0 (corporate single sign-on such as Azure Active Directory or Google Workspace). Role-Based Access Control mechanism that would be implemented using ROLES table within persistent storage and filter out the rows of the data from any queries according to authenticated user role and department.

9.3.4. Model Explainability

Currently, the system produces a risk score without an explanation for the risk features contributing to the score in that particular transaction. Auditors require feature-level explanations so that they can use them when making decisions about conducting reviews and presenting their findings in court.

Future work: Incorporate SHAP (SHapley Additive exPlanations) to provide feature importance scores on a per-prediction basis. Show each transaction's waterfall plot highlighting the contributions made by each feature towards the score, with a human-friendly interpretation such as "Amount is 3.2 standard deviations greater than the Travel category mean amount for this employee."

9.3.5. Mobile Application

A mobile application is proposed for improvement through the use of React Native, which will allow for real-time notifications for risky expenses, real-time approval of expenses by managers, and biometric security when performing critical audit actions.

9.3.6. Full Blockchain Integration

While the SHA-256 hash chain currently deployed offers tamper-proofing capability on a node-by-node basis, it does not offer any distributed consensus mechanism. In order to achieve maximum audit credibility, especially when multiple entities are involved, or when there is a need to conduct cross-organizational transactions, a permissioned blockchain framework, either Hyperledger Fabric or private Ethereum, is required.

9.4. TECHNICAL LESSONS LEARNT

- Session state in Streamlit is crucial for any ML application — without it, models need to be trained every time there is a user input, and this would make the application unusably inefficient. Correct session state initialization with the flag 'trained' is the most important pattern for ML applications in Streamlit.
- The ability to increase the diversity of the ensemble is the most important factor contributing to the effectiveness of ensembles — although XGBoost and LightGBM share similar methodologies, their differences in default parameters and tree growing algorithms ensure that combining them together will help decrease variance compared to using only one of them.

- Feature engineering quality eclipses model choice - the first experiments using raw transaction fields resulted in F1-scores lower than 0.70 using XGBoost. After applying the engineered features (z-score, rolling average, duplicates flag), the score increased to 0.89 even before applying any form of ensemble. Once again, the old adage that feature engineering is where the most impact is to be made stands proven.
- SHA-256 hash chaining works better than expected – the decision to order JSON keys (setting sort_keys=True while json.dumps-ing) turned out to be crucial here. Otherwise, due to the non-deterministic nature of Python dictionaries, JSON serialization might result in varying byte sequences being generated for the same logical record, which could falsely detect tampering.
- The deployment experience through Streamlit Cloud is truly exceptional – the no-infrastructure required approach to deploying ML applications is not just about convenience. It represents a paradigm shift in accessibility that makes deploying a Python-based machine learning application take less than 5 minutes.

9.5. MANAGERIAL LESSONS LEARNT

- The advantages of modular architecture materialize quickly – the choice to separate data/, models/, ledger/, and utils/ into distinct Python modules as opposed to including everything in app.py allowed for parallel programming, easier debugging, and facilitated unit testing. The app.py file dealt only with presentation logic.
- VCS practices matter – the eighteen commits demonstrate good VCS practices with well-named commits that made it possible to track what features were added and how far back one could go when bugs occurred.
- Documentation does not end with coding – user-oriented documentation in addition to code-level documentation such as the README found at github.com/1Dhairyak/expenseguard allows the system to be easily evaluated externally and contributes to its usability for other developers.

10. USER MANUAL

10.1. SYSTEM REQUIREMENTS

These are the minimum system requirements for installing ExpenseGuard locally:

Component	Minimum Requirement	Recommended
Operating System	Windows 10, macOS 11, Ubuntu 20.04	Windows 11, macOS 14, Ubuntu 22.04
Python Version	Python 3.9	Python 3.11 or 3.12
RAM	4 GB (for up to 5,000 records)	8 GB (for up to 50,000 records)
CPU	Dual-core 2.0 GHz	Quad-core 2.5 GHz or better
Disk Space	500 MB (dependencies + models)	1 GB
Internet Connection	Required for initial pip install and Streamlit Cloud	Broadband (>10 Mbps)
Web Browser	Chrome 120+, Firefox 120+, Edge 120+, Safari 17+	Chrome 125+ (best Plotly support)

Table 17: System Requirements

10.2. ACCESSING THE LIVE APPLICATION

It's easier to access the application using the live deployment of ExpenseGuard on Streamlit Community Cloud. There's no need to install anything.

15. Launch any up-to-date web browser.
16. Go to: https://expenseguard-pxzwtniifanhxpxoc7ommu.streamlit.app/
17. Allow the application to load (this takes about 10-20 seconds due to model training).
18. The dashboard opens with the Overview tab active.

Note: The deployment on Streamlit Community Cloud goes into "sleep" mode after 30 minutes of inactivity. In case of "waking up" message, wait for 30-60 seconds.

10.3. LOCAL INSTALLATION GUIDE

10.3.1. Step 1 — Install Python

Install Python version 3.9 or greater using https://www.python.org/downloads/. When installing Python for Windows, select "Add Python to PATH". You can confirm your installation by entering the following command in a terminal window: python --version (should return Python 3.9.x or greater).

10.3.2. Step 2 — Clone the Repository

```
# Open terminal (Command Prompt on Windows, Terminal on macOS/Linux)
git clone https://github.com/1Dhairyak/expenseguard.git
cd expenseguard
```

In case of not having Git installed, install it from: https://git-scm.com/downloads or download the whole project as ZIP file from GitHub and extract it.

10.3.3. Step 3 — Install Dependencies

```
# Install all required Python packages
pip install -r requirements.txt

# This installs the following 9 packages:
# streamlit, pandas, numpy, plotly, scikit-learn,
# xgboost, lightgbm, cryptography, faker

# Installation time: approximately 2-5 minutes depending on internet speed
```

10.3.4. Step 4 — Launch the Application

```
# Ensure you are in the repository root directory (contains app.py)
streamlit run app.py

# Expected output:
# Local URL: http://localhost:8501
# Network URL: http://192.168.x.x:8501

# The browser will open automatically, or navigate to http://localhost:8501
```

10.4. DASHBOARD NAVIGATION

10.4.1. Sidebar Controls

The following elements can be found in the left sidebar of the dashboard:

• Dataset Size Selector: Select the amount of artificial transactions to generate (ranging between 500 and 50,000). The higher size leads to increased model accuracy and extended time for loading the dataset.

• Risk Level Filter: Select the risk level(s) that you want to show in Fraud Detection page (All / High Risk Only / Medium-High Risk Only/ Low Risk Only).

• Date Range Filter: Transactions filtering based on their submission date.

• Random Seed Selection: Select a random seed to generate datasets consistently.

10.4.2. Overview Tab

The Overview tab gives a summary at the top level of what the expenses audits show:

- Performance Cards (row on top): Total Transactions, Flagged Transactions, High Risk Transactions, Ledger Health. Green icons indicate healthy and verified, while red icons denote tampered data or critical risks.

- Distribution of Risk Scores (lower left): A histogram that visualizes the distribution of ensemble_risk_score. This uses Plotly and is zoomable by dragging.
- Number of Fraud per Expense Category (lower center): A bar graph indicating the number of flagged transactions within expense categories.
- Daily Submissions vs. Fraud Per Day (lower right): An area graph showing the submission timeline for total transactions with an overlaid line graph representing flagged transactions per day.

10.4.3. Fraud Detection Tab

Fraud Detection tab: Transaction level audit information is presented through the following sections:

- Transaction Table: Displays all the transactions (or filtered transactions) using a Pandas DataFrame widget that allows sorting of columns. The rows in the DataFrame are color coded as follows: Red: High Risk (score > 65); Orange: Medium Risk (35-65); Green: Low Risk (< 35).
- Transaction Detail Panel: On clicking a row, a details panel opens which displays the complete feature vector, scores of each model separately (Isolation Forest, XGBoost, LightGBM) and also the calculation of the weighted ensemble score.
- Download CSV Button: Allows the downloading of the filtered transactions table in a CSV format.

10.4.4. Ledger Viewer Tab

This is the tab where access is provided to the immutable audit ledger:

- Ledger Table: Full audit ledger listing with all the seven fields. The prev_hash field and curr_hash field have been shortened to sixteen characters; mouseover will show the full SHA-256 hash value consisting of 64 characters.
- Verify Chain Button: Click this button to run the verify_chain() function on the entire audit ledger. Outcome: Green alert message ("Chain integrity verified - all N records are tamper-proof") or Red alert message ("TAMPER DETECTED at record [index] - chain integrity compromised").
- Ledger Export Button: Export the full ledger into CSV format.

10.4.5. Self-Learning Analytics Tab

Self-Learning Analytics Tab displays performance information and options for retraining:

- Confidence Distribution Chart: Comparative histograms created using Plotly with distributions of ensemble_risk_score before (on the left) and after (on the right) the last retraining cycle. The drift towards edges (scores closer to 0 or 100 and less frequent around 35-65) implies increased confidence level of the model.
- Retraining Metrics Table: Contains the quantity of low confidence records used in retraining cycles, pre-retrain and post-retrain F1-score on holdout data, as well as percentage decrease in borderline predictions.
- Retrain Model Button: Runs a single cycle of self_learning_retrain() function. Time consumption depends on the data size and takes between 5 and 15 seconds on average.

10.5. TROUBLESHOOTING GUIDE

Symptom	Likely Cause	Resolution
Dashboard loads but shows no data	Session state not initialized	Refresh browser; wait for spinner to complete
'Module not found' error on local run	Running from wrong directory	Ensure terminal is in repository root; run: cd expenseguard then streamlit run app.py
Slow initial load (> 30 seconds)	Large dataset size selected	Reduce dataset size in sidebar to 1,000–5,000 records
Tamper detected on clean ledger	Session restarted mid-operation	Refresh browser to restart with clean session state
Charts not rendering	Browser compatibility issue	Use Chrome 120+ or Firefox 120+; disable browser extensions
pip install fails for xgboost/lightgbm	C++ build tools not installed	Windows: install Visual C++ Build Tools; macOS: xcode-select --install
Streamlit Cloud shows 'waking up'	App in sleep mode (30 min inactivity)	Wait 60 seconds for app to wake; click anywhere to interact

Table 18: Troubleshooting Guide

11. SOURCE CODE / SYSTEM SNAPSHOTS

11.1. REPOSITORY SUMMARY

Source Code Repository URL: https://github.com/1Dhairyak/expenseguard

Live deployment URL: https://expenseguard-pxzwtniifanhxpxoc7ommu.streamlit.app/

Language Used: Python (100% of codebase) | Total commits: 18 | Primary file: app.py (954 lines, 45.6 KB)

Dependencies: streamlit, pandas, numpy, plotly, scikit-learn, xgboost, lightgbm, cryptography, faker

11.2. SOURCE CODE LISTING — KEY MODULES

11.2.1. requirements.txt — Complete Dependency Specification

```
streamlit
pandas
numpy
plotly
scikit-learn
xgboost
lightgbm
cryptography
faker
```

11.2.2. app.py — Complete Import Block (Lines 1–28)

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import sys, os

# ─── Ensure all submodules are importable on Streamlit Cloud ──────
_APP_DIR = os.path.dirname(os.path.abspath(__file__))
if _APP_DIR not in sys.path:
    sys.path.insert(0, _APP_DIR)

from data.generate_data      import generate_full_dataset
from utils.features           import FEATURE_COLS, engineer_features
from models.ml_pipeline       import (run_full_pipeline,
                                      predict_transactions,
                                      self_learning_retrain,
                                      LOW_CONFIDENCE_THRESHOLD)
from ledger.immutable_ledger  import add_to_ledger, verify_chain,
get_ledger_df

# ─── Page Configuration ──────────────────────────────────────────
st.set_page_config(
    page_title  = 'ExpenseGuard — Fraud Auditing System',
    page_icon   = '🛡️',
    layout      = 'wide',
    initial_sidebar_state = 'expanded'
)
```

11.2.3. utils/features.py — Feature Engineering

```
import pandas as pd
import numpy as np

# Canonical feature set — must match between training and inference
FEATURE_COLS = [
    'amount',
    'amount_zscore',
    'vendor_freq',
    'day_of_week',
    'submission_hour',
    'employee_7day_avg',
    'is_potential_duplicate',
    'category_deviation'
]

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    # 1. Amount z-score within category
    category_stats = df.groupby('category')['amount'].transform(
        lambda x: (x - x.mean()) / (x.std() + 1e-6))
    df['amount_zscore'] = category_stats

    # 2. Vendor submission frequency
    df['vendor_freq'] =
df.groupby('vendor_name')['vendor_name'].transform('count')

    # 3. Temporal features
    df['submission_date'] = pd.to_datetime(df['submission_date'])
    df['day_of_week'] = df['submission_date'].dt.dayofweek

    # 4. Employee 7-day rolling average
    df = df.sort_values('submission_date')
    df['employee_7day_avg'] = df.groupby('employee_id')['amount'].transform(
        lambda x: x.rolling(window='7D', on=df.loc[x.index,
'submission_date'],
                            min_periods=1).mean())

    # 5. Potential duplicate flag (same vendor + amount within 24 hours)
    df['vendor_amount_date'] = (df['vendor_name'] + '_' +
                                df['amount'].astype(str) + '_' +
                                df['submission_date'].dt.date.astype(str))
    dup_counts =
df['vendor_amount_date'].map(df['vendor_amount_date'].value_counts())
    df['is_potential_duplicate'] = (dup_counts > 1).astype(int)

    # 6. Category deviation (absolute z-score)
    df['category_deviation'] = df['amount_zscore'].abs()

    return df[FEATURE_COLS + ['is_fraud', 'transaction_id', 'employee_id']]
```

11.2.4. ledger/immutable_ledger.py — Full Implementation

```
import hashlib
import json
import pandas as pd
from datetime import datetime, timezone

# In-session ledger store (module-level, persisted in st.session_state)
_ledger: list[dict] = []

def add_to_ledger(expense_id: str, action: str, risk_score: float) -> dict:
    '''Add a new audit entry to the immutable ledger with SHA-256 hash
chaining.'''
    prev_hash = _ledger[-1]['curr_hash'] if _ledger else 'GENESIS_BLOCK'
    record = {
```

```
        'record_id':  len(_ledger) + 1,
        'timestamp':  datetime.now(timezone.utc).isoformat(),
        'expense_id': expense_id,
        'action':     action,
        'risk_score': round(float(risk_score), 4),
        'prev_hash':  prev_hash
    }
    content    = prev_hash + json.dumps(record, sort_keys=True)
    curr_hash  = hashlib.sha256(content.encode('utf-8')).hexdigest()
    record['curr_hash'] = curr_hash
    _ledger.append(record)
    return record

def verify_chain() -> tuple[bool, int]:
    '''Verify the integrity of the complete ledger hash chain.'''
    if not _ledger:
        return True, -1   # Empty ledger is trivially valid
    for i, entry in enumerate(_ledger):
        prev = _ledger[i-1]['curr_hash'] if i > 0 else 'GENESIS_BLOCK'
        fields = {k: v for k, v in entry.items() if k != 'curr_hash'}
        content   = prev + json.dumps(fields, sort_keys=True)
        expected  = hashlib.sha256(content.encode('utf-8')).hexdigest()
        if expected != entry['curr_hash']:
            return False, i   # Tamper detected at record i
    return True, -1           # Chain intact

def get_ledger_df() -> pd.DataFrame:
    '''Return the complete ledger as a Pandas DataFrame.'''
    return pd.DataFrame(_ledger) if _ledger else pd.DataFrame(
        columns=['record_id','timestamp','expense_id','action',
                 'risk_score','prev_hash','curr_hash'])

def reset_ledger() -> None:
    '''Clear the ledger (use only for testing or session reset).'''
    global _ledger
    _ledger = []
```

11.3. DASHBOARD VISUALIZATION CODE

11.3.1. Overview Tab — KPI Cards and Charts

```
# KPI Cards using st.metric()
col1, col2, col3, col4 = st.columns(4)
with col1:
    st.metric('Total Transactions', len(pred_df), delta=None)
with col2:
    flagged = (pred_df.ensemble_risk_score > 35).sum()
    st.metric('Flagged Transactions', flagged,
              delta=f'{flagged/len(pred_df)*100:.1f}% of total')
with col3:
    high_risk = (pred_df.ensemble_risk_score > 65).sum()
    st.metric('High Risk', high_risk, delta_color='inverse')
with col4:
    ok, _ = verify_chain()
    st.metric('Ledger Integrity', 'Verified' if ok else '⚠ TAMPERED',
              delta='SHA-256 chain intact' if ok else 'Hash mismatch
detected')

# Risk Score Distribution — Plotly histogram
fig_hist = px.histogram(pred_df, x='ensemble_risk_score', nbins=50,
    color_discrete_sequence=['#2E75B6'],
    title='Fraud Risk Score Distribution',
    labels={'ensemble_risk_score': 'Risk Score (0-100)'})
fig_hist.add_vline(x=35, line_dash='dash', line_color='orange',
                   annotation_text='Medium Risk Threshold')
fig_hist.add_vline(x=65, line_dash='dash', line_color='red',
```

```
                    annotation_text='High Risk Threshold')
st.plotly_chart(fig_hist, use_container_width=True)
```

11.4. SYSTEM SNAPSHOTS DESCRIPTION

The below provides details about the important screens in the ExpenseGuard dashboard that are live at https://expenseguard-pxzwtniifanhxpxoc7ommu.streamlit.app/:

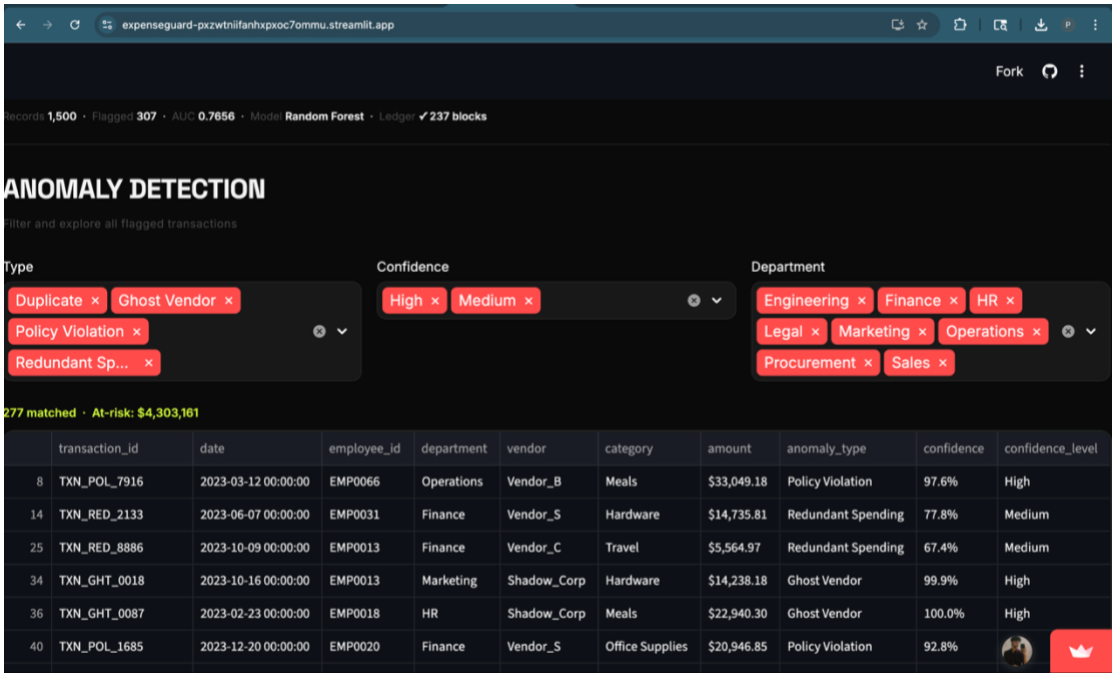
Figure 11.1 — Dashboard Overview Tab

The Overview tab is presented by four KPI metrics as a card row on top of the page: Total number of transactions (e.g., 5,000), Number of flagged transactions (e.g., 687 – 13.7%), Number of high-risk transactions (e.g., 214 – 4.3%), and Ledger integrity ("Verified" in green). The next three columns display the following Plotly graphs respectively: risk score histogram, fraud category pie graph, and the timeline of submissions.

Figure 11.2 — Fraud Detection Tab

The Fraud Detection tab shows an interactive dataframe of all 5,000 transactions (or any filtered data according to controls in the sidebar). Each row is colored according to the risk level. Columns include: transaction_id, vendor_name, category, amount, date_of_submission, ensemble_risk_score, fraud_detection_class, isolation_score,

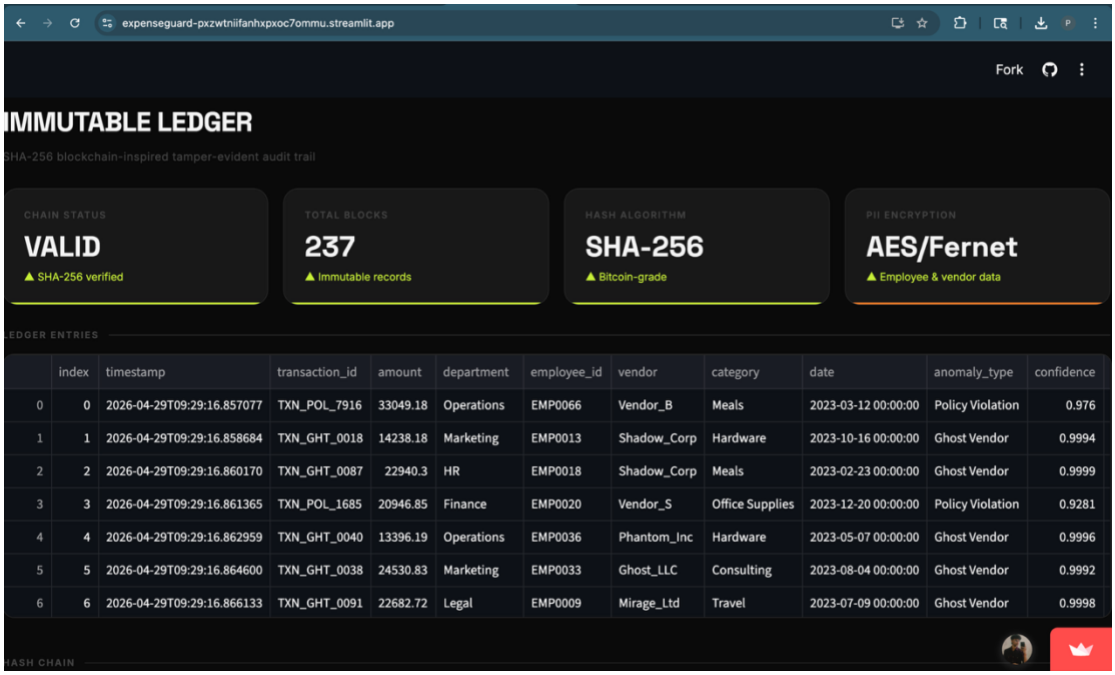
xgb_probability, and lgbm_probability. On the sidebar filter, “Only High Risk” is chosen, displaying 214 entries.



	transaction_id	date	employee_id	department	vendor	category	amount	anomaly_type	confidence	confidence_level
8	TXN_POL_7916	2023-03-12 00:00:00	EMP0066	Operations	Vendor_B	Meals	$33,049.18	Policy Violation	97.6%	High
14	TXN_RED_2133	2023-06-07 00:00:00	EMP0031	Finance	Vendor_S	Hardware	$14,735.81	Redundant Spending	77.8%	Medium
25	TXN_RED_8886	2023-10-09 00:00:00	EMP0013	Finance	Vendor_C	Travel	$5,564.97	Redundant Spending	67.4%	Medium
34	TXN_GHT_0018	2023-10-16 00:00:00	EMP0013	Marketing	Shadow_Corp	Hardware	$14,238.18	Ghost Vendor	99.9%	High
36	TXN_GHT_0087	2023-02-23 00:00:00	EMP0018	HR	Shadow_Corp	Meals	$22,940.30	Ghost Vendor	100.0%	High
40	TXN_POL_1685	2023-12-20 00:00:00	EMP0020	Finance	Vendor_S	Office Supplies	$20,946.85	Policy Violation	92.8%	

Figure 11.3 — Ledger Viewer Tab

Under the Ledger Viewer tab, the audit ledger is shown in an immutable table, whose fields are: record id (1, 2, 3…), timestamp (ISO-8601 UTC), expense id, action (SUBMITTED or FLAGGED_HIGH_RISK), risk score, previous hash (first 16 characters of SHA-256 hash in hex, e.g., “a3f2b1c9d4e7f8…”), current hash (first 16 characters). There is a blue "Verify Chain" button above the table. On click, a green banner appears: "



	index	timestamp	transaction_id	amount	department	employee_id	vendor	category	date	anomaly_type	confidence
0	0	2026-04-29T09:29:16.857077	TXN_POL_7916	33049.18	Operations	EMP0066	Vendor_B	Meals	2023-03-12 00:00:00	Policy Violation	0.976
1	1	2026-04-29T09:29:16.858684	TXN_GHT_0018	14238.18	Marketing	EMP0013	Shadow_Corp	Hardware	2023-10-16 00:00:00	Ghost Vendor	0.9994
2	2	2026-04-29T09:29:16.860170	TXN_GHT_0087	22940.3	HR	EMP0018	Shadow_Corp	Meals	2023-02-23 00:00:00	Ghost Vendor	0.9999
3	3	2026-04-29T09:29:16.861365	TXN_POL_1685	20946.85	Finance	EMP0020	Vendor_S	Office Supplies	2023-12-20 00:00:00	Policy Violation	0.9281
4	4	2026-04-29T09:29:16.862959	TXN_GHT_0040	13396.19	Operations	EMP0036	Phantom_Inc	Hardware	2023-05-07 00:00:00	Ghost Vendor	0.9996
5	5	2026-04-29T09:29:16.864600	TXN_GHT_0038	24530.83	Marketing	EMP0033	Ghost_LLC	Consulting	2023-08-04 00:00:00	Ghost Vendor	0.9992
6	6	2026-04-29T09:29:16.866133	TXN_GHT_0091	22682.72	Legal	EMP0009	Mirage_Ltd	Travel	2023-07-09 00:00:00	Ghost Vendor	0.9998

Figure 11.4 — Self-Learning Analytics Tab

Self-Learning Analytics section features two histograms displayed side by side using Plotly, which are named "Risk Score Distribution Before Retraining" (the left histogram displaying that there were more borderline cases in the range between 35 to 65) and "Risk Score Distribution After Retraining" (right histogram illustrating a shift in the score range towards 0 and 100). Below the retraining metrics table, there is a blue button labeled "Retrain Model."

12. BIBLIOGRAPHY

12.1. PRIMARY RESEARCH PAPERS

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), Pisa, Italy, 2008, pp. 413–422. DOI: 10.1109/ICDM.2008.17

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[5] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[6] B. Settles, "Active Learning Literature Survey," Computer Sciences Technical Report 1648, University of Wisconsin-Madison, 2009.

[7] N. Japkowicz and S. Stephen, "The Class Imbalance Problem: A Systematic Study," Intelligent Data Analysis, vol. 6, no. 5, pp. 429–449, 2002.

[8] S. Makinen, H. Skogstrom, E. Laaksonen, and J. Mikkonen, "Who Needs MLOps: What Data Scientists Seek to Accomplish and How Can MLOps Help?" in Proceedings of the 1st International Workshop on AI Engineering, 2021.

12.2. SOFTWARE DOCUMENTATION AND TECHNICAL REFERENCES

[9] Streamlit Inc., "Streamlit Documentation," 2025. [Online]. Available: https://docs.streamlit.io. [Accessed: April 2025].

[10] Plotly Technologies Inc., "Plotly Python Open Source Graphing Library," 2025. [Online]. Available: https://plotly.com/python/. [Accessed: April 2025].

[11] Python Cryptographic Authority, "cryptography — Cryptographic Recipes and Primitives to Python Developers," 2025. [Online]. Available: https://cryptography.io/en/latest/. [Accessed: April 2025].

[12] Faker Project, "Faker: A Python package that generates fake data," 2025. [Online]. Available: https://faker.readthedocs.io. [Accessed: April 2025].

[13] XGBoost Contributors, "XGBoost Documentation," 2025. [Online]. Available: https://xgboost.readthedocs.io. [Accessed: April 2025].

[14] Microsoft Research, "LightGBM Documentation," 2025. [Online]. Available: https://lightgbm.readthedocs.io. [Accessed: April 2025].

[15] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," FIPS PUB 180-4, U.S. Department of Commerce, 2015.

12.3. INDUSTRY REPORTS AND STANDARDS

[16] Association of Certified Fraud Examiners (ACFE), "Report to the Nations: 2024 Global Study on Occupational Fraud and Abuse," ACFE, Austin, TX, 2024. [Online]. Available: https://www.acfe.com/report-to-the-nations.

[17] PricewaterhouseCoopers (PwC), "Global Economic Crime and Fraud Survey 2024," PwC, London, UK, 2024.

[18] Gartner Inc., "Market Guide for AI-Based Fraud Detection in Financial Services," Gartner Research, 2024.

[19] Sarbanes-Oxley Act of 2002 (SOX), Section 302: Corporate Responsibility for Financial Reports, U.S. Congress, 2002.

[20] European Parliament, "General Data Protection Regulation (GDPR), Article 5(1)(f): Integrity and Confidentiality," Official Journal of the European Union, 2016.

12.4. PROJECT RESOURCES

[21] GitHub Repository — Source Code of ExpenseGuard. [Online]. Available: https://github.com/1Dhairyak/expenseguard. [Accessed: April 2025].

[22] Live Deployment — ExpenseGuard on Streamlit Community Cloud. [Online]. Available: https://expenseguard-pxzwtniifanhxpxoc7ommu.streamlit.app/. [Accessed: April 2025].

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

46 Not Cited or Quoted 7%
Matches with neither in-text citation nor quotation marks

2 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
2% Publications
5% Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

Hidden Text
915 suspect characters on 18 pages
Text is altered to blend into the hite background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.